HDF5 · DESIGN RESEARCH

Streaming in HDF5: beating ADIOS2, not adopting it

HDF5 has five partial answers to streaming and no complete one. The gap is not transport — it is that HDF5 has no **step**. But matching ADIOS2 is the floor, not the goal, and the two constraints turn out to be a single decision: you cannot beat SST while running on top of SST.

TREE EXAMINED	`develop @ d006fb8d3a0`	LAYER
`VOL connector`	REJECTED	`VFD · ADIOS2 dep`
DIFFERENTIATOR	`reader-driven`	

The recommendation, first

Everything below is the argument for this.

> Build streaming as a **VOL connector owning its own wire protocol**, with a thin internal transport abstraction. Add the step as registered *optional operations* rather than new core API. Make the protocol **reader-driven** — that is the thing ADIOS2 structurally cannot do, and it is why the protocol has to be ours.
>
> The two hard pieces a streaming engine needs are *already* in the VOL layer. `H5VL_request_class_t` gives `wait` / `notify` / `cancel` against `H5ES` event sets, and the VOL `read` / `write` callbacks already take a `count` array of datasets. That is ADIOS2's deferred `Put` and its `EndStep` batch, sitting unused.
>
> And the hardest part of M×N redistribution is already in the tree and battle-tested by VDS: `H5Sselect_project_intersection` computes exactly the writer-selection-to-reader-selection projection that redistribution needs.

Why not depend on ADIOS2

Not a political preference. A precondition.

The tempting shortcut is to link ADIOS2's SST as the network engine. SST is mature and solves real problems: RDMA, UCX, MPI, Mercury and WAN transport, rendezvous, queue policy, M×N redistribution, step atomicity. On a pure build-versus-borrow reading, it wins.

It stops winning the moment the goal is to be *better* than ADIOS2 rather than equal to it. Every differentiator in the next section needs something SST's protocol has no room for — a back-channel from reader to writer, a negotiated precision per subscriber, a step cadence that is not globally monotone. You cannot add a reverse channel to a protocol you do not own. So the dependency question and the ambition question have the same answer, and the dependency is ruled out on technical grounds before the organizational ones are even reached.

This is an argument about one category of dependency, not about dependencies. Specialized libraries that are excellent at a bounded job are exactly what this should be built on — nobody should write their own RDMA, their own user-level threading, or their own lossy compressor. What makes ADIOS2 different is not that it is a dependency but that it is a *peer framework which owns the protocol*: it brings its own data model, its own format, its own release cadence, and a wire protocol with no room for the reverse channel. Borrowing a transport takes on none of that.

The next section maps every job SST was doing onto a library that does that one job well — and it turns out the strongest candidates come from a stack HDF5 already has history with.

Where ADIOS2 is genuinely beatable

Ranked by differentiation × how much HDF5 already has. Tags mark what exists versus what must be built.

Reader-driven subscription NEW PROTOCOL

ADIOS2 streaming is push. Writers marshal everything, ship it, and readers select what they want and discard the rest — often the overwhelming majority of it. There is no channel for a reader to say *I need this subvolume, strided by four, at half precision*. `StepDistributionMode=OnDemand` chooses *which* reader gets a step; it cannot change what the step contains.

Let readers declare interest and have writers marshal only what someone subscribed to. This cuts serialization CPU on the writer and bytes on the wire at the same time, and it makes per-subscriber precision negotiation natural: full fidelity to the checkpoint sink, downsampled to the live viz, from one `end_step()`.

Basis: nothing in ADIOS2 does this, and it is the reason the protocol must be ours. Mercury's RPC model already has the reverse direction.

Stream and archive as one object

ONION VFD PRECEDENT

ADIOS2 makes you choose: BP5 to a file, or SST as a stream, with different engines and different code paths. Replaying a stream later means having written it twice.

A step that is simultaneously a live stream element *and* a nameable, re-openable, diffable revision collapses that choice. The onion VFD already models an append-only numbered revision history with `H5FD_ONION_FAPL_INFO_REVISION_ID_LATEST` for "give me the newest" — the semantics are most of the way there already.

Basis: mine `H5FDonion_history.c` and `H5FDonion_index.c` for the history encoding, not the I/O path.

Full data model, and free type conversion

H5T ALREADY DOES IT

ADIOS2 has Variables and Attributes over a flat `/`-delimited namespace and a fixed type set, and it largely assumes writer and reader agree on representation.

HDF5 streaming inherits real groups, compound and enum types, opaque types, references, and dimension scales — strictly more expressive. More usefully, `H5T`'s conversion engine already handles endianness and precision differences between peers, so a heterogeneous stream works without the application knowing. That is close to free, and ADIOS2 has no equivalent.

Basis: existing library capability; the work is not losing it at the connector boundary.

Independent step cadence per group

SEMANTICS WORK

An ADIOS2 step is global and all-variables-together. A mesh written once every hundred field steps forces you to either re-`Put` it every step or work around it with `FirstTimestepPrecious`.

Allow per-group cadences with an explicit dependency: field step *N* references mesh revision *M*. Multiphysics and AMR codes need this, and expressing it honestly is more useful than a global monotone counter.

Basis: pure semantics and manifest design — no transport dependency, decidable in the Phase 0 RFC.

Spill: a third queue-full policy CACHE VOL PRECEDENT

ADIOS2 offers exactly two answers when the step queue fills: `Block`, which stalls the simulation, or `Discard`, which loses science. Both are bad, and the choice between them is forced because the queue lives in memory.

Node-local NVMe makes a third answer available: **spill** the queue to local storage and keep accepting steps. The writer neither stalls nor drops, and a reader that fell behind catches up from disk instead of being abandoned. Every pre-exascale and exascale machine has the hardware, and the Cache VOL already stages HDF5 data on exactly this tier.

Basis: Cache VOL proves the staging pattern; BAKE provides the RDMA-to-local-storage path. Mostly assembly, not invention.

Predicate pushdown on a live stream FORMAT EXTENSION

The payoff of owning the protocol: a reader subscribing with a predicate — *only chunks whose maximum exceeds this threshold* — so filtering happens before the bytes are marshalled, not after they arrive.

Flagged honestly as the expensive one. `H5D_chunk_rec_t` carries `scaled`, `nbytes`, `filter_mask` and `chunk_addr` and **no statistics**, and there is no `H5Q` / `H5X` query API in the tree. This needs per-chunk stats added to the format. It belongs on the roadmap as a reason the architecture is right, not in v1.

Basis: greenfield. `H5Sselect_intersect_block` is the chunk-routing half; the statistics half does not exist yet.

The first four need no new file format and no new dependency. That is what makes this a plan rather than an aspiration.

What ADIOS2 provides — the floor

Parity targets. Five of these eight have no counterpart in HDF5 today.

1. **A step is an atomic transaction.** `BeginStep` / `EndStep` bracket a unit of I/O, and the guarantee is strict: no step is partially dropped, delivered to a subset of ranks, or otherwise divided. Everything else depends on this.

2. **Deferred and sync modes.** `Put(deferred)` registers intent; data moves at `EndStep` or `PerformPuts`. This is what allows batching and zero-copy.
3. **Engines are swappable under one API.** The application does not change when the transport does.
4. **Writer-side flow control.** `QueueLimit` with `QueueFullPolicy` of `Block` (never lose data, stall the simulation) or `Discard` (never stall, lose data), plus `ReserveQueueLimit` for late-arriving readers.
5. **Late joiners are first-class.** `FirstTimestepPrecious` pins step 0 permanently so any reader connecting later still gets the run parameters.
6. **Configurable rendezvous.** `RendezvousReaderCount` decides how many readers `Open()` waits for — including zero — with `OpenTimeoutSecs` and a `RegistrationMethod` that does not assume a shared filesystem.
7. **Reader-side step policy.** `AlwaysProvideLatestTimestep` for monitoring that must not fall behind, and `StepDistributionMode` of `AllToAll`, `RoundRobin` or `OnDemand`.
8. **M×N redistribution.** Writer ranks queue data until it is known which reader rank needs it, so a 4096-rank writer streams to a 16-rank analysis job without an all-to-all.

Items 1, 4, 5, 7 and 8 have no counterpart anywhere in HDF5 today. That is the parity scope, before any of the differentiators.

What the HDF5 tree already has

Five partial answers. Each is genuinely useful and each stops short in a different place.

Classic SWMR

SHARED FS ONLY

One writer, many readers on the same file with no locking. `H5F_ACC_SWMR_WRITE` / `_READ`, `H5Dflush`, `H5Drefresh`, `H5Pset_append_flush`, `H5Pset_object_flush_cb`, and `h5watch` as a working tail-the-dataset tool.

Needs POSIX write ordering — explicitly not NFS or SMB. No structural change while in SWMR mode: no creating or deleting groups, datasets, or attributes. No variable-length data. Requires libver 1.10+, and `H5Fint.c` rejects it alongside a metadata cache image. Readers poll; no notification, no step atomicity.

`VFD SWMR`

PROTOTYPE, UNMERGED

The designed successor: metadata snapshots at a configurable *tick* into a shadow file, readers picking up the newest at API entry. `H5Pset_vfd_swmr_config` with `tick_len` and `max_lag`. Writers may create and delete objects.

Still a shared-filesystem design with no transport. VL data remains incompatible because HDF5 stores it as metadata; VDS poorly integrated; Windows unsupported. Readers exceeding `max_lag` ticks overrun their snapshot. No M×N, no back-pressure.

`Mirror VFD`

WRONG ALTITUDE

Actual network streaming today: `H5FD_MIRROR` ships `OP_OPEN`, `OP_WRITE`, `OP_TRUNCATE`, `OP_SET_EOA`, `OP_LOCK` over TCP, and advertises `H5FD_FEAT_SUPPORTS_SWMR_IO`. Working in-tree socket code to crib.

It replicates byte ranges, not data. The remote side materializes the same random-access file — which is why it works, and why it cannot become a stream. One socket, no MPI or RDMA, no selections, no steps, no back-pressure.

`Onion VFD`

CLOSEST SEMANTICS

An append-only revision history: each commit is a numbered revision and a reader opens a specific one, or `H5FD_ONION_FAPL_INFO_REVISION_ID_LATEST`. This is the foundation for stream-and-archive-as-one-object.

File-local, page-diff granularity, no transport, no notification that a revision landed, no bound on history growth. Mine the history and index encoding, not the I/O path.

`H5S selection projection`

M×N, SOLVED

`H5Sselect_project_intersection` projects one selection through another — precisely the writer-rank-to-reader-rank mapping M×N needs, already exercised by VDS. `H5Sselect_intersect_block` answers "does this chunk touch this reader's selection," the chunk-routing primitive.

Needs wiring into a step manifest and a rank-aggregation topology. The algorithmic core — the part that would have justified borrowing SST — is done and tested.

Async VOL + `H5ES`

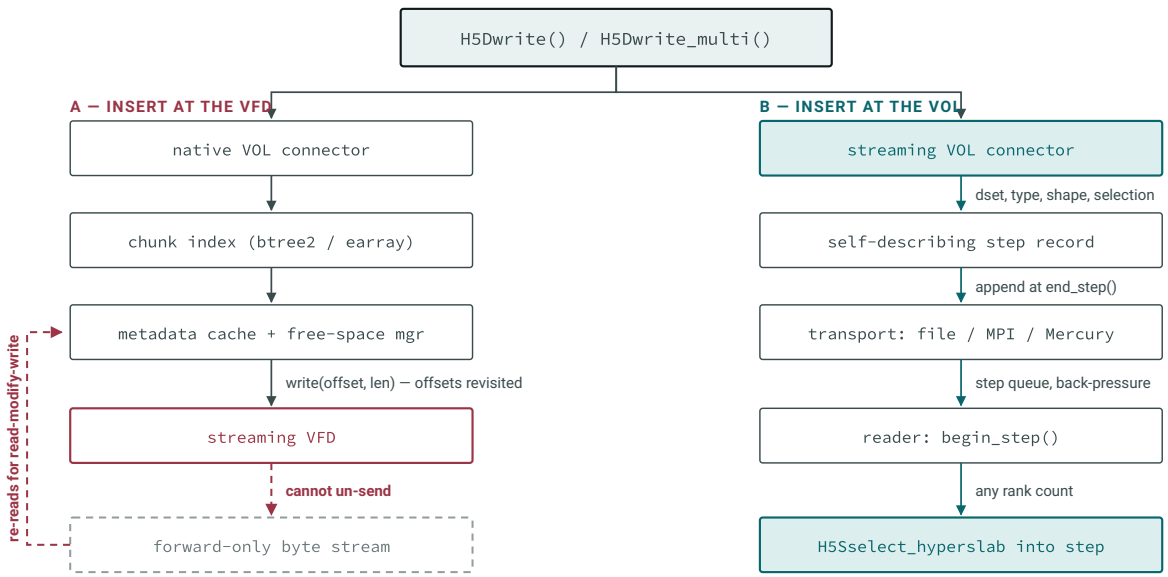
REUSABLE AS-IS

The deferred-operation machinery, already shipped. `H5EScreate` / `H5ESwait` / `H5EScancel` over a connector's `H5VL_request_class_t`, advertised via `H5VL_CAP_FLAG_ASYNC`. Plus `H5Dwrite_multi`, and VOL `read` / `write` callbacks that are natively multi-dataset.

No barrier primitive meaning "complete everything queued before this point" as distinct from "complete everything" — which is what `end_step` needs. `H5ESget_op_counter` exposes the sequence number to key it off.

Why this cannot be a VFD

The cheapest thing to prototype and a guaranteed dead end. HDF5 already learned this once.



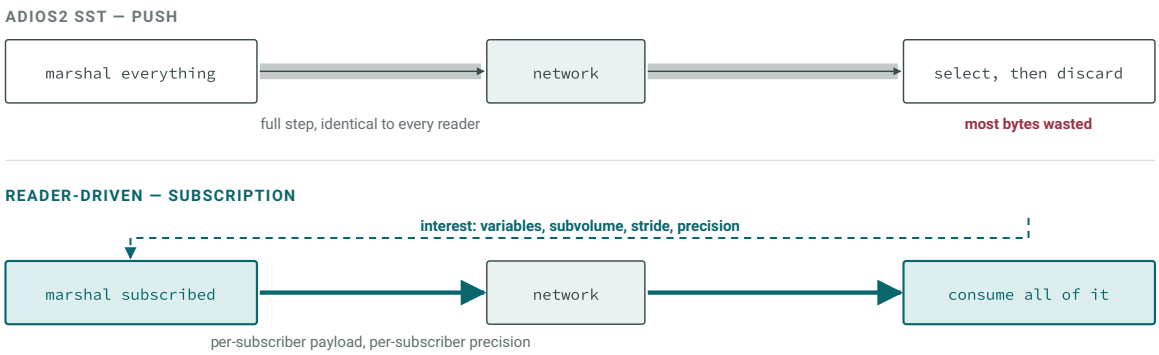
The same write, entering the stack at two different altitudes. On the left the write has already been flattened into byte ranges by the time a driver sees it, and the layers above keep returning to offsets they wrote earlier — the chunk index rebalances, the free-space manager updates, the superblock and object headers are patched, and the metadata cache reads its own writes back. Every one of those is a seek into data a stream has already sent. On the right the write is still *described*, so it serializes forward-only and a reader can select into it arbitrarily.

HDF5 shipped a stream VFD once. `H5FDstream.c`, `H5Pset_fapl_stream` and `H5FD_STREAM` were removed in the 1.8 series, and `H5Fmodule.h` still carries the note that the driver is not available. The failure mode was the diagram's left column, and the mirror VFD is the proof: it only functions because the process on the far end of the socket reconstitutes a fully seekable file.

Two further reasons beyond read-modify-write. M×N needs dataspace knowledge to decide which writer rank's data satisfies which reader's selection, and a driver sees no dataspaces at all. And back-pressure has to be applied where the application's step boundary is visible, so a blocked queue stalls the simulation at a defined point rather than inside an arbitrary metadata flush.

Push versus reader-driven

The differentiator, as a mechanism.



The reverse arrow is the whole difference. Push mode saves writer-side bookkeeping at the cost of serializing and shipping data nobody asked for — the reader's selection happens after the bytes arrive. Adding a declaration of interest before the step moves that filter to the writer, cutting marshalling CPU and wire bytes together, and letting one `end_step()` serve a full-fidelity checkpoint sink and a downsampled live viewer at different precisions. SST's protocol has nowhere to put that arrow, which is why the protocol has to be ours.

What to borrow instead

Every job SST was doing, mapped onto a library that does that one job well.

Dropping SST does not mean writing SST's internals. It means assembling them from components that each solve a bounded problem, so that no single dependency owns the protocol. The Mochi stack covers most of it, and notably Mercury was developed jointly by Argonne *and The HDF Group* — this is not an unfamiliar codebase.

`Mercury`

replaces SST DP

RPC plus bulk RDMA transfer over a network-abstraction layer covering CXI, verbs, TCP and shared memory. Its request/response shape is a *better* substrate for the subscription back-channel than a stream abstraction, because the reverse direction is already first-class rather than bolted on.

`Argobots + Margo`

progress engine

User-level threading and the Mercury glue layer. A step queue needs a background progress thread that does not block the application's compute, and this is precisely what Margo exists to provide. Writing a correct progress engine by hand is a classic way to lose a year.

`SSG`

replaces rendezvous

Scalable Service Groups handles group membership — which is exactly SST's `RegistrationMethod`, `RendezvousReaderCount` and late-joiner discovery problem, generalized and already hardened. No shared filesystem assumption.

`BAKE`

enables queue spill

RDMA-enabled transfer to node-local storage. This is what makes the third queue-full policy possible — see the differentiator above — and it is the same node-local staging ground the Cache VOL already works on.

`ZFP / SZ`

via existing filters

Per-subscriber precision does not need a new compressor. HDF5 already has a dynamically-loaded filter architecture (`H5PL_TYPE_FILTER`) and an established plugin collection, so a negotiated precision maps onto the existing, well-tested filter pipeline rather than new code.

`MPI · sockets · file`

no new deps

The floor, always available. MPI intercommunicators for MPMD-launched coupled codes (parallel HDF5 already requires MPI); the mirror VFD's existing socket code under `H5_HAVE_MIRROR_VFD` for WAN and laptops; and the file engine as the conformance oracle — a stream and its file replay must be bit-identical, which is how the step semantics get tested with no network in the picture.

Build order still starts at the bottom rung, for testing reasons rather than dependency reasons: the file engine is what makes every later transport verifiable.

The plan

Ordered, because each phase's output is the next phase's input. Phase 0 is the one that actually decides the design.

00

RFC ONLY — NO CODE

Pin the step semantics and the wire protocol

This is where the project succeeds or fails, and it is cheap. Each question changes the implementation, so none can be deferred:

- **Is a step file-scoped, or per-group with independent cadence?** ADIOS2 is file-scoped and globally monotone. Going per-group is differentiator four and has to be decided here, because it determines the manifest format.
- **What is an unlimited dimension in a stream?** Recommend the step index becomes an implicit slowest-varying dimension, so per-step `[ny][nx]` writes present as `[nsteps][ny][nx]` in the aggregate view — ADIOS2's `StepsStart` / `StepsCount` in HDF5 terms, landing on either an unlimited dimension zero or a VDS over per-step datasets.
- **The subscription vocabulary.** What can a reader declare interest in — variables, hyperslabs, stride, precision, predicate? Reserve room in the protocol for the predicate case even though it ships much later, because retrofitting a wire format is the expensive kind of mistake.
- **Which subset of the data model is legal inside a step.** Recommend datasets, attributes and groups; exclude variable-length data and references in v1. VFD SWMR hit the same wall for the same underlying reason — HDF5 stores VL data as metadata — so pretending otherwise costs a year.

- **Reuse versus new API.** Strongly prefer reusing `H5Dwrite_multi` and `H5ES`, with step control as VOL optional operations. This is the difference between shipping a plugin and forking the library.

01

IN-TREE, SMALL, UPSTREAMABLE

Core enablement — only what a connector cannot do from outside

- **A sequential-access capability flag.** The `H5VL_CAP_FLAG_*` set in `H5VLpublic.h` runs from `THREADSAFE` through `FLUSH_REFRESH` and has no way to say "random reads and rewrites are unavailable." Without it, every HDF5 tool pointed at a stream fails unrecognizably. This flag is what keeps streaming from becoming a support burden.
- **Canonical optional-operation names registered in-tree** via `H5VLregister_opt_operation`, so independent connectors agree: `begin_step`, `end_step`, `step_status`, `subscribe`. Ship thin public wrappers — `H5Fbegin_step()`, `H5Fend_step()` — that forward to the optional op, so application code is portable and the core library carries no streaming logic.
- **An event-set barrier.** `H5ESwait` waits on everything; `end_step` needs "complete everything inserted before this point." `H5ESget_op_counter` already exposes the sequence number to key off.
- **Promote `H5Pset_append_flush` to an automatic step boundary.** Its callback already fires when a dataset extends past a dimension boundary — a free auto-`end_step` for append-only writers, at almost no cost.

02

OUT OF TREE, ALONGSIDE VOL-ASYNC AND VOL-CACHE

Reference connector: file engine, then MPI

Get the step semantics, the manifest format and the test suite right with no network risk, then add the first real transport. Both rungs are dependency-free, so the whole of parity plus differentiators two, three and four are reachable before any new dependency is even discussed.

The file engine doubles as the correctness oracle for everything after it: replaying a stream into a file and writing that file directly must produce identical results. That invariant is what makes the later transports testable at all.

03

THE REASON TO BUILD THIS

Subscription protocol and M×N

The differentiator, and the phase that justifies owning the protocol. Readers declare interest; writers marshal only what is subscribed; selections are resolved with `H5Sselect_project_intersection` so writer and reader rank counts are fully independent, and `H5Sselect_intersect_block` routes at chunk granularity. Per-subscriber precision falls out of the same negotiation.

Make `begin_step` and `end_step` collective over the writer communicator and aggregate per-rank selections into the step manifest. The subfiling VFD's I/O-concentrator topology in `src/H5FDsubfiling/` is the in-tree precedent for the aggregation pattern, and for how a complex MPI- and thread-using component is structured, built and tested.

04

ADOPTION

Tools, bindings, and the archive story

Streaming without tooling does not get adopted. Add `h5stream` as the counterpart to `bpls`: list steps, tail a live stream, reorganize a stream into a static file. Extend `h5watch`, which already tails a dataset over SWMR, to the step model. Expose the step and subscription calls through h5py, since that is where most analysis-side readers get written.

This is also where differentiator two lands properly: wire step commits to onion revisions so a stream is addressable after the fact, and "replay step 400" becomes an ordinary open rather than a re-run.

05

OPTIONAL, AND THE LONG GAME

RDMA, spill, predicate pushdown, interop bridge

The Mercury and Margo rung, with SSG taking over rendezvous from the file-based registration used earlier — never required to build. Queue spill lands here too, since it wants BAKE and node-local storage; that turns the two-way block-or-discard choice into a three-way one.

Then per-chunk statistics in the format so a subscription can carry a predicate and filtering happens before marshalling — the payoff of having owned the protocol from the start.

Interop belongs here too, as an optional bridge rather than a foundation: ADIOS2 has a Plugin Engine and its HDF5 VOL already exists, so a translation shim is a contained piece of work

once both step models are stable.

Risks and open decisions

What owning the protocol costs, stated plainly.

The integration bill, not the network bill

Borrowing Mercury, Margo and SSG pays down most of what SST would have given free — transport across five interconnects, a progress engine, group membership — because each of those libraries is hardened at that specific job. What does *not* come for free is the layer above: step queue policy, reconnection and reader-eviction semantics, and the subscription protocol itself. That is the genuine cost, and it is also precisely the part that has to be ours for any of the differentiators to exist. Mitigation is still build order — `file` and `mpi` first, so parity and three differentiators land before any RDMA work.

Five optional dependencies is its own kind of cost

Mochi components are individually excellent and collectively a build-system and packaging surface. Keep every rung genuinely optional behind a CMake feature check, keep the no-dependency rung fully functional forever, and resist letting the RDMA path become the only tested one. The Subfiling VFD is the in-tree precedent for how much build complexity a component like this can carry before it becomes a maintenance problem.

Interop is deferred, and someone will ask about it on day one

Dropping SST means HDF5 streams start as an HDF5-only ecosystem, and ADIOS2 readers cannot consume them. Have the answer ready: an optional bridge in Phase 5, not a dependency in the foundation. Being honest that v1 does not interoperate is better than implying it might.

Every HDF5 tool assumes random access

`h5dump` on a stream will do something unhelpful. The Phase 1 capability flag is the entire mitigation — it lets tools detect and report the situation instead of failing obscurely, and it must land before any connector ships.

Predicate pushdown is a format change, not a feature flag

`H5D_chunk_rec_t` carries no statistics and there is no query API in the tree. Keep it visible as a roadmap payoff and out of any v1 commitment, or it will be read as a promise. What Phase 0 owes it is only wire-format room, which is cheap.

Someone will propose a streaming VFD
By far the cheapest thing to demo, it will appear to work on a write-once-read-never test, and it cannot be made correct. HDF5 removed `H5FD_STREAM` in the 1.8 series for exactly this reason. Recording that history in the RFC is worth the paragraph it costs.

Sources

1. ADIOS2 — SST engine parameters
2. ADIOS2 — Supported engines
3. ADIOS2 — HDF5 API support through VOL
4. Eisenhauer et al. — Streaming Data in HPC Workflows Using ADIOS
5. Mercury — RPC and bulk transfer for HPC
6. Mochi — composable HPC data services
7. Ross et al. — Mochi: Composing Data Services for HPC
8. libfabric / OFI
9. HDF5 filter plugins
10. HDF5 SWMR User's Guide
11. VFD SWMR User's Guide
12. HDF5 Asynchronous I/O VOL connector
13. HDF5 Cache VOL connector